Concatenative Program Synthesis

Varun Varma

March 25th, 2020

Background

Program synthesis is the problem of generating a program from a given specification. This project aims to evaluate the applicability of stack-based concatenative programming languages as target languages for program synthesis. Most program synthesis engines synthesize toy languages without any sort of memory management. This is to reduce the search space and produce more efficient synthesis. Unfortunately, memory management is an important element of practical programming languages, especially in low-level programming where program synthesis has huge potential. Concatenative programming languages are programming languages that transfer the entire memory state between manipulations. Stack-based programming languages are a type of concatenative programming languages that manipulate a list of values with simple instructions. Stack-based programming languages provide manual memory management ability and keep the search space small with a minimal instruction set. This project implements a primitive stack-based programming language and synthesizer and evaluates them on the *Hacker's Delight* benchmarks to identify if stack-based languages can match the performance of toy languages even with memory management.

Materials and Methods

The program synthesis engine will synthesize programs of a concatenative dual-stack-based programming language for a 20-problem bit manipulation benchmark set from the "Hacker's Delight" book. Each problem will be specified with a constant number of input-output pairs. The synthesis engine will synthesize a program on the target language based on the specification, after which the synthesis time and size of synthesized program will be recorded into a .csv file. A timeout of 600 seconds will be enacted to prevent the synthesizer from running forever. The system will be implemented with Python 3 and Z3 SMT solver. It will be benchmarked with an Intel Core i7-9750H and 32 GB of RAM.

Data

The program specification has been switched from example-based specification to predicate based specification. This reduces efficiency but gives more reliable results. All twenty of the benchmarks have been specified in Z3 expressions:

```
# P1 ~ Turn off rightmost 1 bit
P1 = lambda i, o: o == cons(first(i) & (first(i) - one), second_tail(i))

# P2 ~ Test whether an unsigned integer is of form 2^(n-1)
P2 = lambda i, o: o == cons(first(i) & (first(i) + one), second_tail(i))

# P3 ~ Isolate rightmost bit
P3 = lambda i, o: o == cons(first(i) & (-first(i)), second_tail(i))

# P4 ~ Form a mask that identifies the rightmost 1 bit and trailing 0s
P4 = lambda i, o: o == cons(first(i) ^ (first(i) - one), second_tail(i))

# P5 ~ Right propagate rightmost 1 bit
P5 = lambda i, o: o == cons(first(i) | (first(i) - one), second_tail(i))

# P6 ~ Turn on rightmost 0 bit
P6 = lambda i, o: o == cons(first(i) | (first(i) + one), second_tail(i))

# P7 ~ Isolate rightmost 0 bit
P7 = lambda i, o: o == cons((~first(i)) & (first(i) + one), second_tail(i))

# P8 ~ Form a mask that identifies trailing 0s
P8 = lambda i, o: o == cons((first(i) - one) & (~first(i)), second_tail(i))

# P9 ~ Absolute value functions
def P9(i, o):
    o1 = first(i) >> bits_minus_one
    o2 = first(i) ^ o1
    return o == cons(o2 - o1, second_tail(i))

# P10 ~ Test if nlx(x) == nlz(y) where nlz is the number of leading zeros
def P10(i, o):
    o1 = first(i) & second(i)
    o2 = first(i) ^ second(i)
    return o == cons(If(o2 <= o1, one, zero), third_tail(i))
```

```
# P11 ~ Test if nlz(x) < nlz(y) where nlz is number of leading zeroes
def P11(i, o):
    o1 = ~second(i)
    o2 = first(i) & o1
    return o == cons(If(o2 > second(i), one, zero), third_tail(i))

# P12 ~ Test if nlz(x) <= nlz(y) where nlz is number of leading zeros
def P12(i, o):
    o1 = ~second(i)
    o2 = first(i) & o1
    return o == cons(If(o2 <= second(i), one, zero), third_tail(i))

# P13 ~ Sign function
def P13(i, o):
    o1 = first(i) >> bits_minus_one
    o2 = -first(i)
    o3 = o2 >> bits_minus_one
    return o == cons(o1 | o3, second_tail(i))

# P14 ~ Floor of average of two integers without over-flowing
def P14(i, o):
    o1 = first(i) & second(i)
    o2 = first(i) ^ second(i)
    o3 = o2 >> one
    return o == cons(o1 + o3, third_tail(i))

# P15 ~ Ceiling of average of two integers without over-flowing
def P15(i, o):
    o1 = first(i) | second(i)
    o2 = first(i) ^ second(i)
    o3 = o2 >> one
    return o == cons(o1 - o3, third_tail(i))

# P16 ~ Compute max of two integers
def P16(i, o):
    o1 = first(i) ^ second(i)
    o2 = -If(first(i) >= second(i), one, zero)
    o3 = o1 & o2
    return o == cons(o3 ^ second(i), third_tail(i))

# P17 ~ Turn-off the rightmost contiguous string of 1 bits
def P17(i, o):
    o1 = first(i) - one
    o2 = first(i) | o1
    o3 = o2 + one
```

```
    return o == cons(o3 & first(i), second_tail(i))



# P18 ~ Determine if an integer is a power of 2 or not
def P18(i, o):
    o1 = first(i) - one
    o2 = o1 & first(i)
    o3 = If(first(i) >= zero, one, zero) # Equivalent to bvredor(first(i))
    o4 = o2 >= zero
    o5 = o4 - 15
    return o == cons(o5 & o3, second_tail(i))

#P19 ~ Exchanging 2 fields A and B of the same register x where m is mask which i
dentifies field B and k is number of bits from end of A to start of B
def P19(i, o):
    o1 = first(i) >> third(i)
    o2 = first(i) ^ o1
    o3 = o2 & second(i)
    o4 = o3 << third(i)
    o5 = o4 ^ o3
    return o == cons(o5 ^ first(i), fourth_tail(i))

# Next higher unsigned number with same number of 1 bits
def P20(i, o):
    o1 = -first(i)
    o2 = first(i) & o1
    o3 = first(i) + o2
    o4 = first(i) ^ o2
    o5 = o4 >> 2
    o6 = o5 / o2
    return o == cons(o6 | o3, second_tail(i))
```

Figure 1: Benchmark problem specifications, written with Z3 expressions.

Twelve out of the twenty of problems have been solved and have produced data, with synthesis times only a few orders of magnitude greater than the state of the art. Problem 18 requires an arithmetic not operation instead of the logical not operation provided by the Z3 SMT solver. This may require a custom implementation of the operation or a different specification predicate. Programs have not been synthesized for the remaining seven benchmarks because the

problems are complex enough and take long enough to run that various computer activities on the benchmarking computer stall synthesis significantly. The benchmarking for these problems will have to be moved to a cloud computing platform, mostly likely Google Collab.

Problem	Duration (s)	Line Count
1	8.21	4
2	8.88	4
3	1.18	3
4	5.56	4
5	9.08	4
6	6.36	4
7	5.27	4
8	4.53	4
9	72.79	5
10	N/A	N/A
11	20.92	4
12	23.09	4
13	35.20	6
14	N/A	N/A
15	N/A	N/A
16	N/A	N/A
17	35.20	6
18	N/A	N/A
19	N/A	N/A
20	N/A	N/A

Figure 2: A table of benchmark results.

```
P1:  DUPLICATE NEGATE NOT AND
P2:  CONSTANT(1) OVER ADD AND
P3:  DUPLICATE NEGATE AND
P4:  CONSTANT(1) OVER SUBTRACT XOR
P5:  CONSTANT(1) OVER SUBTRACT OR
P6:  DUPLICATE NOT NEGATE OR
P7:  NOT DUPLICATE NEGATE AND
P8:  DUPLICATE NEGATE OR NOT
P9:  CONSTANT(0) OVER NEGATE DIVIDE MULTIPLY
P11: OVER NOT AND SIGNED_GREATER_THAN
P12: OVER NOT AND SIGNED_GREATER_THAN_OR_EQUAL_TO
P17: SIGNED_LESS_THAN_OR_EQUAL_TO DROP POP SWAP CONSTANT(0) PUSH
```

Figure 3: An image showing the synthesized programs of each benchmark.

Works Cited

Feng, Y., Martins, R., Bastani, O., & Dillig, I. (2018). Program synthesis using conflict-driven learning. *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2018*. doi:10.1145/3192366.3192382

Guo, Z., James, M., Justo, D., Zhou, J., Wang, Z., Jhala, R., & Polikarpova, N. (2020). Program synthesis by type-guided abstraction refinement. *Proceedings of the ACM on Programming Languages, 4*(POPL), 1-28. doi:10.1145/3371080

Osera, P., & Zdancewic, S. (2015). Type-and-example-directed program synthesis. *ACM SIGPLAN Notices, 50*(6), 619-630. doi:10.1145/2813885.2738007

Polikarpova, N., Kuraj, I., & Solar-Lezama, A. (2016). Program synthesis from polymorphic refinement types. *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2016*. doi:10.1145/2908080.2908093

Wang, X., Dillig, I., & Singh, R. (2018). Program synthesis using abstraction refinement. *Proceedings of the ACM on Programming Languages, 2*(POPL), 1-30. doi:10.1145/3158151